

Server-Factory-Core-Framework

Агульны рухавік кожнага Server Factory.

Змест

Core Framework — гэта фрэймворк Kotlin, які ляжыць у аснове сямейства інструментаў разгортвання Server Factory. Ён забяспечвае агульны рухавік і абстракцыі, на якіх будуецца такія праекты, як Mail Server Factory, каб кожная «фабрыка» выкарыстоўвала адзін правераны ў баях фундамент замест таго, каб паўтараць прымітывы разгортвання.

Кароткае апісанне

Агульны фрэймворк Kotlin, які ляжыць у аснове экасістэмы Server Factory. Ён забяспечвае агульны рухавік разгортвання, абстракцыі падлучэнняў і механізм крокаў усталявання, якія выкарыстоўваюцца далейшымі фабрыкамі (Mail Server Factory, Web Service Factory, SonarQube Factory і іншыя).

Падрабязнае апісанне

Core Framework — гэта тая няўлоўная інжынерная частка, якая робіць магчымым усё сямейства Server-Factory: паўторна выкарыстоўваны рухавік, на якім будуюцца кожны асобны прадукт «фабрыкі» (Mail Server Factory, Web Service Factory, SonarQube Factory, Caching Proxy Factory). Падыход Server Factory з'яўляецца дэкларатыўным — карыстальнік апісвае інфраструктуру, якую ён хоча, у выглядзе канфігурацыі, а фабрыка інтэрпрэтуе гэты апісанне, каб усталяваць і ініцыялізаваць праграмае забеспячэнне на мэтавай сістэме. І нават у **Core Framework** жыве механізм, агульны для гэтага шаблону: абстракцыі падлучэнняў і транспарту, якія дасягаюць кожнага тыпу мэтавай сістэмы, мадэль крокаў усталявання, якая кадзіруе як праграмае забеспячэнне разгортваецца, і агульная інфраструктура, якую ў адваротным

выпадку кожнай фабрыцы давялося б пісаць самастойна. Гэта адказ на структурнае пытанне, з якім рана ці позна сутыкаецца любы інструментальны ланцужок з некалькіх прадуктаў: дзе павінен знаходзіцца агульны рухавік? І калі адказ на гэтае пытанне дадзены правільна адзін раз, гэта захоўвае цэласнасць сямейства замест таго, каб яно распадалася на чатыры крыху розныя разгортвальнікі. Цэнтралізацыя гэтага ў адным фрэймворку Kotlin дазваляе сямейству пазбегнуць дублявання лагікі разгортвання ў прадуктах і захоўвае аднастайнасць паводзін: паляпшэнне тыпу падлучэння ці прымітыву усталявання ў **Core Framework** прыносіць карысць кожнай далейшай фабрыцы. Ён амаль цалкам напісаны на Kotlin (прыкладна 990 Кб кода Kotlin з тонкім пластом Shell), што адлюстроўвае яго ролю як бібліятэкі кода, а не калекцыі скрыптоў. Далейшыя рэпазіторыі спасылаюцца на яго як на кананічную залежнасць (Parallels-Utils, Qemu-Utils, Utils і пакеты Definitions усе звяртаюцца да рэпазіторыя **Core Framework** як да цэнтра экасістэмы). Яго README наўмысна мінімальны — гэта інфраструктура для іншых праектаў, версіі якой кіруюцца праз `version.txt/version_code.txt` — і ён з'явіўся раней за пазнейшую працу AI, што робіць яго часткай сталай спадчыны DevOps-інструментарыя арганізацыі.

Навошта мы яго стварылі

Кожнаму інструменту разгортвання патрэбны адны і тыя ж асновы: спосабы падлучэння да мэтавых сістэм і крокі для ўсталявання/наладкі праграмага забеспячэння. Калі б гэта паўтаралася для кожнага прадукту, паводзіны сталі б фрагментаванымі, а колькасць памылак павялічылася б. **Core Framework** цэнтралізуе гэта, каб усе фабрыкі выкарыстоўвалі адзін надзейны рухавік.

Чаму гэта змяняе правілы гульні

Гэта найбольш уплывовы пункт ва ўсім сямействе: калі тып падлучэння ўдасканалены ці прымітыў усталявання палепшаны тут, гэтае паляпшэнне і магчымасці распаўсюджваюцца на ўсе фабрыкі адначасова, таму ўвесь інструментальны ланцужок павялічвае сваю эфектыўнасць дзякуючы адной інвестыцыі. Гэта філасофія «ствары адзін раз, выкарыстоўвай усюды», прымененая там, дзе яна прыносіць найбольшую карысць — на ўзроўні фундаменту аўтаматызацыі інфраструктуры, дзе выпраўленне ў правільным месцы выпраўляе ўсё, што ідзе далей.

Змест

Што новага

- Адзін зноў выкарыстоўваемы фрэймворк для правізіянінгу, які абстрагуе логіку падлучэння і ўстаноўкі.
- Чыстае падзяленне паміж рухавіком (Асноўны фрэймворк) і фабрыкамі, спецыфічнымі для прадуктаў.
- Дыстрыбуцыя з фіксаванай версіяй (version.txt/version_code.txt) для паўторнага выкарыстання.

Праблемы і рашэнні

- **Пазбяганне дублявання логікі правізіянінгу:** вырашана шляхам вынясення агульнай механікі ў адзіны фрэймворк, які выкарыстоўваецца ўсімі фабрыкамі.
- **Аднастайныя паводзіны ў розных прадуктах:** вырашана з дапамогай агульных абстракцый, каб тыпы падлучэнняў і этапы дзейнічалі аднолькава паўсюль.
- **(НЕПРАВЕРАНА):** пэўныя ўнутраныя API не дакументаваны ў публічным README; разглядайце дэталі інтэрфейсаў як неправераныя за межамі “агульнага фрэймворка, які выкарыстоўваецца фабрыкамі”.

Тэхнічны стэк (чаму + як)

- **Kotlin** — увесь фрэймворк (~990К байтаў); мова сямейства Server Factory.
- **Shell** — мінімальныя дапаможныя скрыпты.
- **Gradle** — інструменты зборкі (адпавядае выкарыстанню ./gradlew у сямействе).

Заўвага: GitHub пазначае рэпазіторый як форк у арганізацыі Server-Factory. Не засяроджаны на AI; пададзены як аснова інструментарыя правізіянінгу.